KV-Mesh: A Prototype for Cache-Affinity Routing and Tiered Key-Value State Management in Distributed LLM Serving

Ayaan Shaheer

Independent Researcher | Software prototype: https://github.com/AyaanShaheer/kv-mesh

Manuscript status: prototype/system-design study; do not submit until marked experiments are executed and the related-work review is updated.

Abstract

Repeated prompts are common in multi-turn dialogue, retrieval-augmented generation (RAG), and agent workflows. When inference requests are dispatched without regard to cache locality, a request may reach a worker that cannot reuse state previously computed by another worker, reducing the opportunity to avoid repeated prefill work. This paper presents KV-Mesh, an early distributed-serving prototype that combines a Go/gRPC gateway, Python worker processes, heartbeat-based worker-state reporting, prompt-fingerprint-based routing, and a C++/pybind11 host-memory pool. The gateway maintains a time-bounded worker registry and preferentially dispatches a request to a live worker whose advertised cache index contains the request fingerprint; otherwise it falls back to round-robin selection. The prototype also models a two-tier cache lifecycle in which entries are evicted from a small simulated GPU-resident cache into a host-memory byte pool and fetched back on demand. A source-code inspection shows that the present worker emits a synthetic response and uses synthetic byte strings rather than real model-generated tokens and GPU-resident KV tensors. Accordingly, the system is evaluated here as a routing and state-management prototype, not as a demonstrated acceleration of production LLM inference. We define a reproducible evaluation plan comparing cache-affinity routing with round-robin and least-loaded baselines across repeated-prefix workloads, worker counts, telemetry delays, and cache capacities. The intended contribution is an openly inspectable prototype and measurement framework for studying when mesh-level cache locality is useful, what stale metadata costs, and which implementation changes are required before real KV-cache offloading can be claimed.

Keywords: large language model serving; KV cache; prefix caching; cache affinity; distributed routing; tiered memory; gRPC; systems evaluation

1. Introduction

Autoregressive transformer inference typically processes an input prompt before generating output tokens. The attention key/value (KV) state produced during processing can be retained and reused when subsequent requests share a compatible prefix. Reuse can avoid recomputing portions of the prompt, but it depends on cache availability, cache identity, and the inference engine’s semantics. Prefix caching is already supported by modern serving systems, including vLLM, and research systems such as LMCache explore broader KV-state storage, reuse, and movement [1, 2]. The problem considered in this work is narrower: when multiple independent workers serve related requests, a gateway that ignores cache locality may send a request to a worker that lacks reusable state even when another healthy worker has it.

KV-Mesh is an exploratory prototype for coordinating such workers. The implementation uses a Go gateway to track worker identifiers, addresses, heartbeat timestamps, cache fingerprints, and reported utilization. A Python worker reports its state through gRPC and exposes a streaming generation method. A C++ extension stores byte payloads in a host-memory map and supplies offload/prefetch-shaped operations. The design is deliberately small enough to inspect and extend, making it suitable for a controlled systems experiment.

This paper does not claim that cache-aware routing, KV-cache reuse, or host-memory offloading is new in itself. Instead, it asks whether a lightweight mesh control plane that combines cache-affinity routing with worker liveness metadata can improve routing locality under constrained and changing worker state, and what evidence is required to validate that claim against simple baselines.

1.1 Research questions

- RQ1: Under repeated-prefix workloads, how does cache-affinity routing change the cache-hit rate relative to round-robin routing?
- RQ2: How do heartbeat period, metadata staleness, and worker churn affect routing accuracy and failed requests?
- RQ3: What routing overhead does the gateway add as worker count and request rate increase?
- RQ4: When host-memory retrieval is modeled or implemented, under which cache sizes and reuse distances is offload/prefetch preferable to recomputation?

1.2 Contributions

- An open prototype architecture joining a Go/gRPC routing gateway with Python workers and a pybind11/C++ byte-pool interface.
- A cache-affinity decision path that prefers live workers advertising a matching request fingerprint and uses round-robin fallback on a reported miss.
- A concrete experimental protocol that separates control-plane routing behavior from actual model-inference performance and identifies measurements needed before making latency or throughput claims.
- A source-level limitation analysis and an implementation roadmap for transitioning from simulated state to genuine engine-managed KV tensors.

2. Background and Related Work

2.1 KV caching and prefix reuse

A transformer serving engine may preserve attention keys and values so that a later request with a matching prefix avoids recomputing the corresponding prompt tokens. vLLM documents automatic prefix caching using hashes over token blocks and preceding-prefix context [1]. A crucial distinction for KV-Mesh is that its current fingerprint is computed over the entire raw prompt string. This is not equivalent to token-block prefix matching: it recognizes exact string equality only, is sensitive to formatting and tokenization differences, and does not by itself establish that two requests have a safely reusable token prefix. A production design should use an engine-compatible cache key, including model/version, tokenizer, relevant adapter, cache format, and block/prefix identity.

2.2 KV-cache storage and transfer

LMCache describes a cache layer that enables reuse and movement of KV state across requests and engines, including tiered storage and transfer mechanisms [2]. Other work, including CacheGen, studies KV-cache compression and streaming [3]. These systems show that moving cache state is an established research area. KV-Mesh's current C++ component should therefore be described precisely as a host-memory byte pool prototype: it copies an input byte string into a C++ vector and returns that byte string on retrieval. The Python worker supplies a synthetic 1 MiB payload when an entry is evicted. The present code does not allocate CUDA memory, access a GPU tensor, use pinned host memory, or perform a PCIe transfer. These are implementation opportunities, not experimentally established features.

2.3 Serving-system scheduling

Serving research has studied workload-aware allocation and separation of prompt prefill from token decoding. DistServe, for example, disaggregates these phases to optimize serving goodput under latency constraints [4]. KV-Mesh addresses a different control point: it selects a worker based on advertised cache state before forwarding a request. These approaches are complementary in principle, but no integration or comparison with DistServe is implemented in the current repository.

3. System Design

3.1 Components and request path

The system consists of (i) a Go gateway exposing gRPC services, (ii) Python worker servers, (iii) a protocol-buffer interface for generation and heartbeat reporting, and (iv) a C++ extension that keeps host-memory byte entries. A client sends a generation request to the gateway. The gateway asks the router for a worker, opens a gRPC client stream to that worker, and relays response chunks to the client. Workers periodically submit a heartbeat with their ID, reachable address, utilization estimate, and fingerprints currently resident in the simulated fast-tier list.

Component	Current responsibility	Evidence in implementation
Go gateway	Receives heartbeat updates; selects worker; proxies streamed response	gateway/main.go; gateway/router/router.go
Routing table	Maintains worker records and their advertised fingerprints; rejects entries last seen 10 s ago	Router.UpdateWorkerState; Router.GetBestWorker
Python worker	Reports every 3 s; models two resident cache slots; yields fixed synthetic text chunks	worker/main.py
C++/pybind11 pool	Stores and returns byte vectors keyed by 64-bit fingerprint	worker/cpp_extension/ memory_manager.cpp
Protocol buffers	Defines GenerateStream, SendHeartbeat, and request/response fields	proto/kvmesh.proto

3.2 Cache-affinity routing policy

Let W(t) be the set of workers whose last heartbeat was observed within the 10-second liveness window. For request prompt p, the router computes a 64-bit fingerprint h(p). If there exists an active worker w whose advertised set Cw contains h(p), the router returns the first such worker encountered. Otherwise, it selects a worker using a rotating index over the active-worker slice. This is a simple membership-based affinity policy, not a globally optimal scheduler: the code does not currently score queue depth, utilization, estimated prefill cost, cache-entry size, or the number of reusable prefix tokens.

Algorithm 1: Current routing policy (descriptive pseudocode)

1. Build the active-worker list using heartbeat age < 10 seconds.
2. If the list is empty, return “no active workers”.
3. Compute h = HashPrompt(prompt).
4. If an active worker advertises h, return that worker.
5. Otherwise, advance a rotating index and return that worker.

3.3 Cache identity and state

The current Go and Python hash helpers both take the first 16 hexadecimal digits of an MD5 digest of the UTF-8 prompt string and parse them as an unsigned 64-bit integer. Matching functions make the current prototype interoperable, but a 64-bit truncated fingerprint carries collision risk and does not bind the key to model or tokenizer identity. For a real implementation, use a collision-resistant cache key or verify full key material, and ensure that cache residency comes from the inference engine rather than a separately maintained approximation.

3.4 Tiered-memory prototype

The worker models a two-slot fast tier as a Python list and applies FIFO eviction. When it evicts an entry, it constructs a 1 MiB byte string and passes it to the C++ extension. The extension copies the bytes into a vector in an unordered map. A prefetch retrieves and erases the saved bytes. This tests the control-flow shape of offload and retrieval, but not the bandwidth, synchronization, correctness, fragmentation, or latency of GPU-to-CPU movement. The README’s PCIe and GPU-VRAM descriptions should be read as intended architecture rather than literal behavior in the current code.

3.5 Liveness and telemetry

Workers attempt to send heartbeats every three seconds, and the router excludes a worker whose most recently received heartbeat is at least ten seconds old. The gateway replaces the stored worker record on every heartbeat. This provides a minimal soft-state registry. The implementation does not include heartbeat sequence numbers, out-of-order update protection, explicit deregistration, bounded connection reuse, or a separate metrics/control API. Since the registered address currently defaults to loopback, the prototype should be run on a single host unless address configuration is added.

4. Implementation Status and Assumptions

A research paper should distinguish implemented code, modeled behavior, and planned production integration. Table 2 makes this separation explicit.

Capability	Status established by source inspection	Claim permitted in this paper
Multi-worker gRPC gateway and streaming relay	Implemented in Go; workers expose gRPC stream.	Prototype relays synthetic streamed output.
Heartbeat registry and TTL filtering	Implemented in Go/Python at 3 s heartbeat and 10 s expiry window.	Can study liveness metadata and expiry behavior.
Exact-prompt fingerprint affinity	Implemented using truncated MD5-derived 64-bit fingerprints and advertised resident entries.	Can evaluate placement behavior for exact prompt repeats.
Round-robin fallback	Implemented for fingerprint misses.	Can compare routing decisions against simple baselines.
Real model inference	Not established; worker yields fixed synthetic text chunks.	Do not report real-model TTFT, throughput, or output quality.
Real GPU KV tensor cache	Not implemented in inspected worker.	Do not claim cache reuse skips transformer prefill.
GPU-to-CPU PCIe transfer	Not implemented; byte strings copied to C++ host memory.	Describe as simulated byte-pool offload/prefetch.
Dynamic VRAM usage	Reported value is list length divided by two-slot simulated capacity.	Call this a simulated occupancy indicator, not actual VRAM telemetry.

5. Evaluation Methodology

No performance measurements are included in this draft because no experimental logs or raw benchmark results were provided. This section specifies the minimum evaluation required to support the proposed claims. The final paper should include environment details, exact commit SHA, scripts, raw CSV output, analysis code, and confidence intervals.

5.1 Baselines

- Round-robin: route each request to the next live worker without examining cache state.
- Least-loaded: route to the live worker reporting the lowest queue depth or, in the initial simulator only, lowest modeled occupancy. State clearly that occupancy is not queue depth.
- Cache-affinity (KV-Mesh): use a valid cache match first; otherwise use round-robin fallback.
- Optional oracle: choose a worker with a true matching cache entry using a simulator-maintained ground-truth state; use only to estimate an upper bound, not as a deployable baseline.

5.2 Workloads

Workload	Construction	Purpose	Primary metric
Exact repeats	Replay the same prompt repeatedly, with controlled inter-arrival times.	Tests exact-key locality.	Affinity hit rate; routing overhead

Hot-set / Zipf-like	Draw prompts from a skewed set where a small hot subset repeats often.	Approximates recurring system prompts and shared context IDs.	Hit rate; request completion rate
Uniform / low reuse	Choose prompt IDs uniformly from a large set.	Tests affinity benefit when reuse is low.	Overhead; load distribution
Worker churn	Terminate/restart workers during replay.	Tests stale registrations and failure recovery.	Routing errors; recovery time
Heartbeat delay/loss	Inject heartbeat delay, drop, or jitter in a test harness.	Measures stale cache-state effect.	False-affinity rate; stale-state duration
Capacity pressure	Vary simulated fast-tier slots and request reuse distance.	Exercises FIFO eviction and host byte-pool path.	Tier hit rate; copied bytes; operation latency

5.3 Experimental factors and repetitions

Start with 1, 2, 4, and 8 worker processes where machine capacity permits; cache capacities of 2, 8, 32, and 128 entries; heartbeat intervals of 1, 3, and 5 seconds; and a 10-second stale-worker TTL (then vary TTL separately). For each cell, warm up the process, replay at least 10,000 requests or run long enough to expose steady-state behavior, and repeat each condition at least five times with recorded seeds. These are suggested design values, not results. Avoid changing multiple factors in a way that prevents attribution.

5.4 Metrics and definitions

- Routing cache-hit rate = requests routed to a worker advertising a matching key / total eligible requests.
- True cache-hit rate = requests whose selected worker actually retains the reusable state, verified from ground truth. Report separately from the router's advertised-hit rate.
- Routing overhead = gateway decision time measured from receipt of a request to worker selection; report median, p95, and p99.
- Request failure rate = failed or timed-out requests / submitted requests.
- Stale-affinity rate = requests routed based on a matching advertisement when the state is absent at request execution time / advertised-hit requests.
- Worker imbalance = coefficient of variation of assigned requests and, in a real inference test, queue depth or GPU utilization.
- Memory path latency = measured time for store/retrieval in the byte-pool microbenchmark; do not label this PCIe or GPU-transfer latency.
- For a future real-model integration: TTFT, time per output token (TPOT), end-to-end latency, throughput, GPU utilization, bytes transferred, and output correctness under the same prompt set and model configuration.

5.5 Statistical analysis

Report median and p95/p99 for latency distributions, with bootstrap 95% confidence intervals for differences between paired runs where appropriate. Report effect sizes and raw sample counts, not only percentage changes. Include all seeds and configuration parameters. Separate simulator/control-plane metrics from real inference metrics; do not infer TTFT improvements from routing-hit rate alone.

6. Results

Not yet measured.

The repository currently demonstrates the intended sequence through console output and synthetic responses, but no quantitative performance data are supplied. This section must remain unfilled until the experiments above have been run. The manuscript must not claim a percentage reduction in TTFT, higher throughput, PCIe transfer speed, model-scaled capacity, or production readiness without measurements directly supporting each statement.

6.1 Results table template

Policy	Workload	Advertised hit %	True hit %	Routing p95 (ms)	Failure %

[fill after run]	[workload]	—	—	—	—
[fill after run]	[workload]	—	—	—	—
[fill after run]	[workload]	—	—	—	—
[fill after run]	[workload]	—	—	—	—
[fill after run]	[workload]	—	—	—	—

7. Discussion

7.1 What the prototype can currently establish

Source inspection supports the existence of a cache-affinity decision path, heartbeat-derived worker records, time-based exclusion of stale workers, a rotating fallback, a streaming gateway proxy, and a C++ byte pool. A controlled simulator can evaluate the behavior of these mechanisms without requiring a large model or multiple GPUs. This is a useful engineering stepping stone because it isolates policy choices from the complexity of a full serving stack.

7.2 Why affinity may help—and when it may not

Affinity can improve locality when requests reuse exactly identified state and the target worker has capacity to serve them. However, a cache match should not automatically dominate load balancing: a very busy worker may impose queueing delays larger than the cost of a cache miss elsewhere. Stale heartbeats can direct requests to entries that have already been evicted. When prompts have low reuse, cache lookup overhead may provide little benefit. A stronger policy should therefore compare estimated avoided prefill cost against queueing delay, transfer cost, memory pressure, and the fraction of prefix tokens reusable.

7.3 Threats to validity

- Simulation gap: generated text and host-memory bytes are synthetic and do not model transformer execution, CUDA memory, or PCIe bandwidth.
- Cache-key validity: exact raw-prompt hashes do not identify token-level reusable prefixes or encode model/tokenizer compatibility.
- Stale state: heartbeats are periodic snapshots; entries may change between reports, and the implementation does not reconcile a cache miss back to the gateway immediately.
- Selection fairness: the router iterates over a Go map, so the order of workers is not a stable placement policy; the round-robin index is also updated while an RLock is held, which is a concurrency-safety problem under simultaneous requests.
- Transport configuration: the current code uses insecure gRPC credentials and loopback addresses; production deployment would require secure transport, reachable advertised addresses, deadlines, and lifecycle management.
- No benchmark data: the repository README provides example logs, not a reproducible experimental dataset.

8. Conclusion and Future Work

KV-Mesh is an early prototype exploring a small distributed control plane for exact-prompt cache affinity and a tiered-memory lifecycle abstraction. Its Go, Python, Protocol Buffers, and C++ components provide a concrete basis for controlled experimentation. At the same time, the current worker is a simulation: it does not connect cache entries to real model KV tensors, does not run real LLM generation, and does not execute actual GPU/PCIe transfers. The next steps are to fix routing concurrency and deterministic selection, strengthen cache identity, add testable telemetry and ground-truth simulator instrumentation, and run repeatable routing experiments. Only after the gateway integrates with an inference engine’s real cache API and moves genuine KV tensors should end-to-end TTFT, throughput, or GPU-memory claims be evaluated.

Acknowledgments

No external funding or institutional support is claimed in this draft. Add acknowledgments only where applicable.

References

[1] vLLM Project, “Automatic Prefix Caching,” vLLM documentation. Available: https://docs.vllm.ai/en/latest/design/prefix_caching/ (accessed 13 Oct. 2026).

[2] Y. Cheng et al., “LMCache: An Efficient KV Cache Layer for Enterprise-Scale LLM Inference,” arXiv:2510.09665, 2025. Available: https://arxiv.org/abs/2510.09665.

[3] Y. Liu et al., “CacheGen: KV Cache Compression and Streaming for Fast Large Language Model Serving,” Proceedings of the ACM SIGCOMM 2024 Conference, pp. 38–56, 2024.

[4] Y. Zhong et al., “DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving,” 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24), pp. 193–210, 2024. Available: https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin.

[5] A. Shaheer, “KV-Mesh: A High-Performance Distributed LLM Inference Router with Prefix-Cache-Aware Routing & Heterogeneous Tiered KV-Memory Offloading,” source repository, https://github.com/AyaanShaheer/kv-mesh (accessed 13 Oct. 2026).

Appendix A. Recommended Engineering Work Before Submission

- **P0 — correctness:** Protect routing state with a write lock when advancing the round-robin index or use an atomic/indexing design; add deterministic worker ordering; add unit tests for TTL expiry, matching, no-worker errors, and concurrent requests.
- **P0 — honest implementation labels:** Update README terminology to call the worker response, two-slot cache, VRAM utilization, and C++ byte-pool behavior simulated until replaced by real engine integration.
- **P1 — simulator and reproducibility:** Add a workload driver, seeded trace generation, policy switch, run configuration file, structured JSON/CSV logs, and a script that calculates metrics/plots confidence intervals.
- **P1 — better keys:** Replace raw-string prompt hash with engine-derived token-block/prefix identifiers; include model revision, tokenizer revision, adapters, and other compatibility boundaries; avoid treating a hash match as sufficient without safe key design.
- **P1 — liveness and load awareness:** Add request deadlines, worker connection reuse, explicit error handling, queue depth and cache-entry size telemetry, and a policy that trades locality against queueing delay.
- **P2 — real KV integration:** Integrate with a supported inference engine cache interface; implement actual CPU/GPU tensor movement, synchronization, capacity accounting and correctness tests, then measure transfer vs recomputation on real hardware.
- **P2 — paper-grade comparisons:** Compare against engine-native prefix caching and an appropriate KV cache layer where feasible; state version/configuration; publish scripts, data, logs, and limitations.

Appendix B. Minimum Reproducibility Checklist

- Record repository commit SHA, operating system/kernel, CPU, RAM, GPU model/VRAM if used, CUDA/driver versions, Go/Python/compiler versions, and exact dependency set.
- Publish one command to build the C++ extension and one command to start the gateway and workers.
- Record workload trace and random seed, number of warmup requests, measurement duration, concurrency level, request timeout, and repetition count.
- Archive raw per-request events (request ID, prompt key, selected worker, advertised hit/miss, true hit/miss, queue depth if present, timestamps, error).
- Separate results from the synthetic simulator and the real LLM-engine experiment in both tables and prose.
- Include unsuccessful and negative results; document deviations from the preregistered or planned protocol.